

Answer Key

Capstone 1: Financial Fraud Detection

SQL Tasks:

3. Data Loading:

- 3.1 Create a schema named **finance**, set it as the default schema, and create tables using **cc_data_dec19.csv** and **location_data.csv**

Output:

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following queries:

```
1 # Create Schema named finance
2 CREATE SCHEMA finance;
3
4 # Set Schema as default
5 USE finance;
6
7 # Make sure both the files are imported by following commands
8 SELECT * from finance.cc_data_dec19;
9
10 SELECT * from finance.location_data;
```

The Results window displays the output of the queries. The first query, `SELECT * from finance.cc_data_dec19;`, returned 1000 rows. The second query, `SELECT * from finance.location_data;`, returned 1 row.

id	index	trans_date_trans_time	cc_num	merchant	category	amt	first	last	gender	street
910366	29-12-2019 04:03	3546897637165770	fraud_Reichert, Shanahan and Hayes	shopping_net	59.62	Kayla	O'Brien	F	7921 Robe	
923619	31-12-2019 15:47	4010002218955870	fraud_Baumbach, Hordkewicz and Walsh	shopping_pos	9.52	Jennifer	Bishop	F	589 Matth	
879455	22-12-2019 05:38	3592325941359220	fraud_Friesen-Stamm	gas_transport	75.86	Ashley	Robinson	F	1007 Coltr	
869204	19-12-2019 23:26	4906628655840910000	fraud_Bernhard, Grant and Langworth	shopping_pos	4.03	Charles	Moreno	M	838 Frankl	
816673	08-12-2019 07:07	3551717884704730	fraud_Erick Group	grocery_net	47.4	Sara	Harrie	F	507 Tamar	

The Action Output window shows the following actions:

#	Time	Action	Message	Duration / Fetch
47	20:19:53	SELECT COUNT(*) from finance.cc_data_dec19 LIMIT 0, 1000	1 row(s) returned	0.047 sec / 0.000 sec
48	20:24:12	SELECT * from finance.cc_data_dec19 LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec

4. Data Exploration with SQL:

- 4.1 Calculate the total number of transactions in the **cc_data_dec19** table

MySQL Workbench

test x

File Edit View Query Database Server Tools Scripting Help

Navigator

Filter objects

SCHEMAS

finance

Tables

cc_data_dec19

location_data

Views

Stored Procedures

Functions

sys

Administration Schemas

Information

No object selected

SQL File 2* SQL File 3*

Limit to 1000 rows

12 ## Data Exploration with SQL

13

14 # Calculate the total number of transactions in the cc_data table.

15 • SELECT COUNT(*) AS total_transactions FROM cc_data_dec19;

16

17

18

19

20

21

22

23

Result Grid

Filter Rows:

Export:

Wrap Cell Content: 15

total_transactions

42308

Result 8 x

Read Only Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
50	20:26:13	SELECT COUNT(*) AS total_transactions FROM cc_data_dec19 LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
51	20:26:18	SELECT COUNT(*) AS total_transactions FROM cc_data_dec19 LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

4.2 Identify the top 10 most frequent merchants in the **cc_data_dec19** table

MySQL Workbench

test x

File Edit View Query Database Server Tools Scripting Help

Navigator

Filter objects

SCHEMAS

finance

Tables

cc_data_dec19

location_data

Views

Stored Procedures

Functions

sys

Administration Schemas

Information

No object selected

SQL File 2* SQL File 3*

Limit to 1000 rows

16

17 #Identify the top 10 most frequent merchants in the cc_data table.

18 • SELECT merchant, COUNT(*) AS transaction_count

19 FROM cc_data_dec19

20 GROUP BY merchant

21 ORDER BY transaction_count DESC

22 LIMIT 10;

23

24

25

26

27

Result Grid

Filter Rows:

Export:

Wrap Cell Content: 15

Fetch rows:

merchant	transaction_count
fraud_Kilback LLC	158
fraud_Cormier LLC	124
fraud_Mraz-Herzog	114
fraud_Kuhn LLC	113
fraud_Relly, Heaney and Cole	113
fraud_Coleman LLC	113

Result 9 x

Read Only Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
51	20:26:18	SELECT COUNT(*) AS total_transactions FROM cc_data_dec19 LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
52	20:27:23	SELECT merchant, COUNT(*) AS transaction_count FROM cc_data_dec19 GROUP BY merc...	10 row(s) returned	0.187 sec / 0.000 sec

Object Info Session

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

4.3 Find the average transaction amount for each category of transactions in the **cc_data_dec19** table

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'finance' expanded, showing 'cc_data_dec19'. The main editor contains the following SQL query:

```
23
24 # Find the average transaction amount for each category of transactions in the cc_data table.
25 • SELECT category, AVG(amt) AS average_transaction_amount
26 FROM cc_data_dec19
27 GROUP BY category;
```

The 'Result Grid' shows the following data:

category	average_transaction_amount
shopping_net	85.20275200989482
shopping_pos	79.39641920251857
gas_transport	63.44431129476586
grocery_net	54.37440414507777
health_fitness	54.37848014440433
total_merch	68.11045699999999

The 'Output' pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
52	20:27:23	SELECT merchant, COUNT(*) AS transaction_count FROM cc_data_dec19 GROUP BY merc...	10 row(s) returned	0.187 sec / 0.000 sec
53	20:28:42	SELECT category, AVG(amt) AS average_transaction_amount FROM cc_data_dec19 GROU...	14 row(s) returned	0.219 sec / 0.000 sec

4.4 Determine the number of fraudulent transactions and the percentage of total transactions that they represent

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'finance' expanded, showing 'cc_data_dec19'. The main editor contains the following SQL query:

```
28
29 # Determine the number of fraudulent transactions and the percentage of total transactions that they represent.
30 • SELECT
31     SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END) AS fraudulent_transactions,
32     (SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END) / COUNT(*)) * 100 AS percentage_of_total
33 FROM cc_data_dec19;
```

The 'Result Grid' shows the following data:

fraudulent_transactions	percentage_of_total
180	0.4255

The 'Output' pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
53	20:28:42	SELECT category, AVG(amt) AS average_transaction_amount FROM cc_data_dec19 GROU...	14 row(s) returned	0.219 sec / 0.000 sec
54	20:30:28	SELECT SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END) AS fraudulent_transaction...	1 row(s) returned	0.046 sec / 0.000 sec

4.5 Join the **cc_data_dec19** and **location_data** tables to identify the latitude and longitude of each transaction

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
34
35 # Join the cc_data and location_data tables to identify the latitude and longitude of each transaction.
36 * SELECT cc.*, ld.lat, ld.long
37 FROM cc_data_dec19 cc
38 JOIN location_data ld ON cc.cc_num = ld.cc_num;
```

The Result Grid shows the following data:

Index	trans_date_trans_time	cc_num	merchant	category	amt	first	last	gender	street
788115	01-12-2019 18:04	302388000000000	fraud_Adams-Barrows	health_fitness	44.57	Danielle	Yu	F	5395 Colon Burgs
795807	02-12-2019 22:41	213153000000000	fraud_Fadel Inc	health_fitness	12.06	Rebecca	Conley	F	181 Moreno Light
835482	12-12-2019 10:59	475570000000000	fraud_Doyle Ltd	grocery_pos	62.56	Robert	Evans	M	01892 Patricia Vis
795657	02-12-2019 22:13	300747000000000	fraud_Padberg-Rogahn	entertainment	115.9	Kelsey	Richards	F	889 Sarah Station
815103	07-12-2019 13:56	676373000000000	fraud_Kalar, Sauer and Bauchi	shopping_mer	5.86	Utki	Mandana	F	5645 Alton Tefar

The Action Output shows the following results:

#	Time	Action	Message	Duration / Fetch
54	20:30:28	SELECT SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END) AS fraudulent_transaction...	1 row(s) returned	0.046 sec / 0.000 sec
55	20:31:33	SELECT cc.*, ld.lat, ld.long FROM cc_data_dec19 cc JOIN location_data ld ON cc.cc_num = ...	1000 row(s) returned	0.000 sec / 0.000 sec

4.6 Identify the city with the highest population in the **location_data** table

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
39
40 # Identify the city with the highest population in the location_data table.
41 * SELECT
42 city,
43 MAX(city_pop) AS highest_population
44 FROM
45 cc_data_dec19
46 GROUP BY
47 city
48 ORDER BY
49 highest_population DESC limit 1;
```

The Result Grid shows the following data:

city	highest_population
Houston	2906700

The Action Output shows the following results:

#	Time	Action	Message	Duration / Fetch
54	20:30:28	SELECT SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END) AS fraudulent_transaction...	1 row(s) returned	0.046 sec / 0.000 sec
55	20:31:33	SELECT cc.*, ld.lat, ld.long FROM cc_data_dec19 cc JOIN location_data ld ON cc.cc_num = ...	1000 row(s) returned	0.000 sec / 0.000 sec
56	20:32:38	SELECT city, MAX(city_pop) AS highest_population FROM cc_data_dec19 GROUP ...	1 row(s) returned	0.235 sec / 0.000 sec

4.7 Find the earliest and the latest transaction dates in the **cc_data_dec19** table

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
# Find the earliest and latest transaction dates in the cc_data table.
SELECT
  MIN(STR_TO_DATE(trans_date_trans_time, '%d-%m-%Y %H:%i')) AS earliest_transaction_date,
  MAX(STR_TO_DATE(trans_date_trans_time, '%d-%m-%Y %H:%i')) AS latest_transaction_date
FROM cc_data_dec19;
```

The Result Grid shows the following output:

earliest_transaction_date	latest_transaction_date
2019-12-01 00:02:00	2019-12-31 23:59:00

The Output pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
55	20:31:33	SELECT cc.`, id lat. id long FROM cc_data_dec19 cc JOIN location_data ld ON cc.cc_num = ...	1000 row(s) returned	0.000 sec / 0.000 sec
56	20:32:38	SELECT city, MAX(city_pop) AS highest_population FROM cc_data_dec19 GROUP ...	1 row(s) returned	0.235 sec / 0.000 sec
57	20:34:03	SELECT MIN(STR_TO_DATE(trans_date_trans_time, '%d-%m-%Y %H:%i')) AS earliest_tran ...	1 row(s) returned	0.109 sec / 0.000 sec

5. Data Aggregation with SQL:

5.1 What is the total amount spent across all transactions in the **cc_data_dec19** table?

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
## Data Aggregation with SQL
# What is the total amount spent across all transactions in the cc_data table?
SELECT SUM(amt) AS total_amount_spent FROM cc_data_dec19;
```

The Result Grid shows the following output:

total_amount_spent
2966148.8299999796

The Output pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
59	20:37:36	SELECT SUM(amt) AS total_amount_spent FROM cc_data_dec19 LIMIT 0, 1000	1 row(s) returned	0.062 sec / 0.000 sec
60	20:37:42	SELECT SUM(amt) AS total_amount_spent FROM cc_data_dec19 LIMIT 0, 1000	1 row(s) returned	0.031 sec / 0.000 sec
61	20:37:44	SELECT SUM(amt) AS total_amount_spent FROM cc_data_dec19 LIMIT 0, 1000	1 row(s) returned	0.031 sec / 0.000 sec

5.2 How many transactions occurred in each category in the **cc_data_dec19** table?

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
62
63 # How many transactions occurred in each category in the cc_data table?
64 • SELECT category, COUNT(*) AS transaction_count
65 FROM cc_data_dec19
66 GROUP BY category;
```

The Results grid shows the following data:

category	transaction_count
shopping_net	3234
shopping_pos	3812
gas_transport	4356
grocery_net	1544
health_fitness	2770
kids_pets	3633
entertainment	3960

The Action Output pane shows the execution of the query:

#	Time	Action	Message	Duration / Fetch
60	20:37:42	SELECT SUM(amt) AS total_amount_spent FROM cc_data_dec19 LIMIT 0, 1000	1 row(s) returned	0.031 sec / 0.000 sec
61	20:37:44	SELECT SUM(amt) AS total_amount_spent FROM cc_data_dec19 LIMIT 0, 1000	1 row(s) returned	0.031 sec / 0.000 sec
62	20:38:48	SELECT category, COUNT(*) AS transaction_count FROM cc_data_dec19 GROUP BY category	14 row(s) returned	0.172 sec / 0.000 sec

5.3 What is the average transaction amount for each gender in the **cc_data_dec19** table?

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
67
68 # What is the average transaction amount for each gender in the cc_data table?
69 • select
70 gender,
71 AVG(amt) as average_transaction_amount
72 from cc_data_dec19
73 group by gender
74 order by gender;
```

The Results grid shows the following data:

gender	average_transaction_amount
F	69.89404608811988
M	70.36812646981976

The Action Output pane shows the execution of the query:

#	Time	Action	Message	Duration / Fetch
64	20:40:05	select gender, AVG(amt) as average_transaction_amount from cc_data_dec19 group by gender	2 row(s) returned	0.140 sec / 0.000 sec
65	20:41:00	select gender, AVG(amt) as average_transaction_amount from cc_data_dec19 group by gender	2 row(s) returned	0.156 sec / 0.000 sec

5.4 Which day of the week has the highest average transaction amount in the **cc_data_dec19** table?

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'finance' expanded, showing tables like 'cc_data_dec19'. The main editor contains a SQL query to find the day of the week with the highest average transaction amount from the 'cc_data_dec19' table. The query is as follows:

```
77
78 # Which day of the week has the highest average transaction amount in the cc_data table?
79 • SELECT
80     DAYNAME(STR_TO_DATE(trans_date_trans_time, '%d-%m-%Y %H:%I')) AS day_of_week,
81     AVG(amt) AS average_transaction_amount
82 FROM cc_data_dec19
83 GROUP BY day_of_week
84 ORDER BY average_transaction_amount DESC
85 LIMIT 1;
86
87
```

The 'Result Grid' shows the following data:

day_of_week	average_transaction_amount
Monday	74.23743150298786

The 'Output' pane at the bottom shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
65	20:41:00	select gender, AVG(amt) as average_transaction_amount from cc_data_dec19 group by ge...	2 row(s) returned	0.156 sec / 0.000 sec
66	20:42:45	SELECT DAYNAME(STR_TO_DATE(trans_date_trans_time, '%d-%m-%Y %H:%I')) AS day_...	1 row(s) returned	0.109 sec / 0.000 sec

